

NMAP FOR PENTESTER



HEX VALUE
OF **FLAGS**



Contents

Overview	3
Introduction	3
Lab Environment	3
Understanding TCP Control Flags	4
Decimal to Hexadecimal Number System	5
The --scanflags Option	6
Single-Flag Probes	7
NULL — 0x00 (no bits set)	7
FIN — 0x01 (bit 0)	7
SYN — 0x02 (bit 1)	8
RST — 0x04 (bit 2)	9
PSH — 0x08 (bit 3)	10
ACK — 0x10 (bit 4)	10
URG — 0x20 (bit 5)	11
Combining Flags with Bitwise OR	12
FIN + PSH + URG — 0x29	12
FIN + SYN + PSH — 0x0B	14
FIN + RST + PSH — 0x0D	15
FIN + SYN + RST + PSH — 0x0F	16
Consolidated Results	18
Mitigation Strategies	18
Drop illegal and anomalous flag combinations	19
Detect, rate-limit, and reduce exposure	19
Conclusion	19





Overview

This article demonstrates how to take direct control of the TCP control-bit field with Nmap's **--scanflags** option, and how to prove the result on the wire with Wireshark. It builds each of the standard flags from its raw value, fires every flag and several flag combinations at a live target, and reads the target's reaction to infer port state and firewall posture. The work begins with single-bit probes — NULL, FIN, SYN, RST, PSH, ACK, and URG — then escalates to multi-flag bytes, finishing with 0x0F, where the entire low nibble of the flag field is set. The closing sections translate the findings into concrete defensive measures and a summary of what the exercise reveals about anomalous TCP traffic.

Introduction

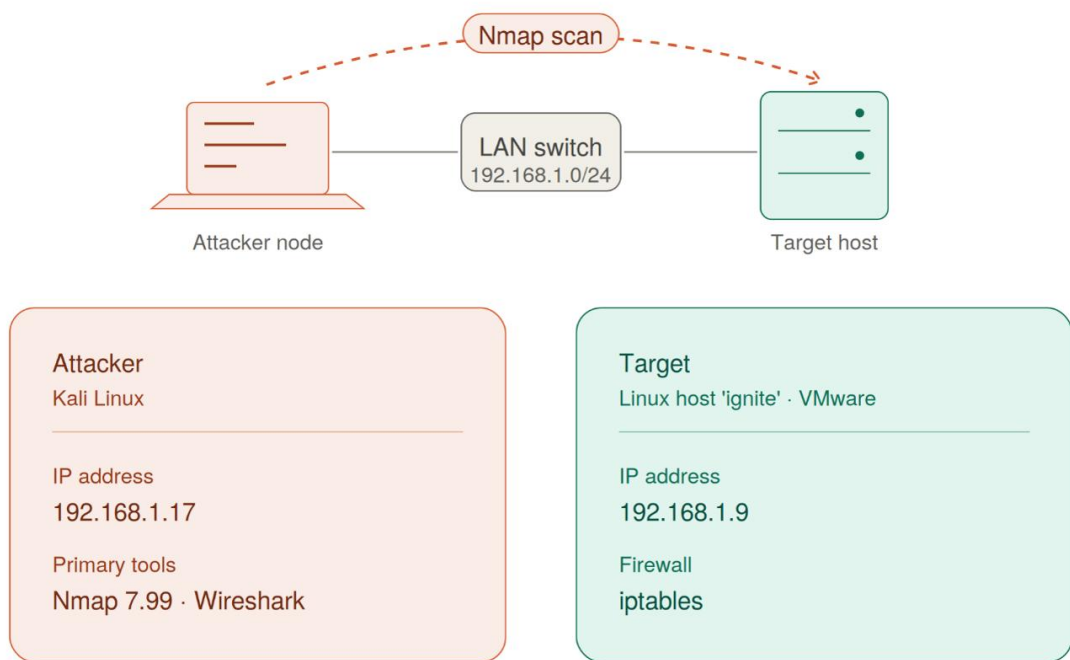
Every TCP segment carries a control-bit field whose individual flags drive the connection lifecycle: SYN opens a handshake, ACK acknowledges data, FIN closes a session gracefully, RST aborts it, and PSH, URG, ECE, CWR, and NS govern delivery and congestion signalling. Standard Nmap scan types send fixed flag patterns, but **--scanflags** removes that constraint and lets the analyst place any combination of bits into a probe. That control is valuable for two reasons. First, it exposes how a host and its firewall respond to unusual or outright illegal flag combinations, which standard scans never generate. Second, it underpins firewall and intrusion-detection testing as well as operating-system fingerprinting, because different TCP stacks answer the same malformed packet in different ways.

The sections that follow move methodically from theory to practice: a reference for every flag, a short primer on reading hexadecimal, the **--scanflags** syntax, the single-flag probes, the multi-flag combinations, a consolidated results table, and finally the mitigations a defender should apply.

Lab Environment

A Kali Linux attacker probes a single Linux target across a flat /24 segment. Nmap 7.99 crafts and sends every probe, while Wireshark captures the exchange and decodes the flag field of each segment. Every capture is filtered on **ip.addr == 192.168.1.9** so that only traffic to and from the target appears. The table below summarises the setup.





Understanding TCP Control Flags

Because each flag occupies a single bit of the control field, its value is a distinct power of two: FIN sits at bit 0 (1), SYN at bit 1 (2), and the sequence continues up to NS at bit 8 (256). This one-bit-per-flag design is what makes flag arithmetic so simple later — any combination is just the sum, or bitwise OR, of the individual values.

The reference below maps every flag to its decimal value, hexadecimal value, binary pattern, and bit position — the four notations the rest of the article moves between.



FLAGS	DECIMAL VALUE	HEXADECIMAL VALUE
NULL	0	0x00
FIN	1	0x01
SYN	2	0x02
RST	4	0x04
PSH	8	0x08
ACK	16	0x10
URG	32	0x20
ECE	64	0x40
CWR	128	0x80
NS	256	0x100

Decimal to Hexadecimal Number System

Understanding number systems is fundamental in computer science and cybersecurity. The decimal system, also known as base-10, is the standard number system used in everyday life, utilising digits 0 through 9. However, computers and digital systems often work with hexadecimal (base-16), which provides a more compact way to represent binary data.

The conversion between decimal and hexadecimal follows a straightforward pattern for the first sixteen values. Decimal numbers 0 through 9 maintain identical representations in hexadecimal, appearing as 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9, respectively. The distinction becomes apparent at decimal 10, where hexadecimal transitions to alphabetic characters to represent values that would otherwise require two digits.

In hexadecimal notation, decimal 10 is represented as 'A', decimal 11 as 'B', decimal 12 as 'C', decimal 13 as 'D', decimal 14 as 'E', and decimal 15 as 'F'. This single-character representation for values up to 15 is what makes hexadecimal particularly efficient for computing applications. Each hexadecimal digit represents exactly 4 binary bits, making it an ideal intermediate notation for reading and writing binary data in a human-friendly format.

For cybersecurity professionals and penetration testers, hexadecimal notation appears frequently in memory addresses, network protocols, exploit payloads, and debugging operations. Tools such as Wireshark, debuggers, and hex editors display data in hexadecimal





format because it provides a balance between the compactness needed for large datasets and the readability required for analysis. Understanding this fundamental conversion is essential for analysing packet captures, crafting exploits, and interpreting memory dumps during security assessments.

DECIMAL NUMBER	HEXADECIMAL NUMBER
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	A
11	B
12	C
13	D
14	E
15	F

The --scanflags Option



Nmap exposes the control-bit field through `--scanflags`. The option takes a numeric value (decimal or hexadecimal) or a symbolic string of flag names, and Nmap then writes those exact bits into every probe it sends. Pairing it with `-p` pins the probe to a single port, which keeps each capture short and unambiguous. The general form is:

```
nmap -p<port> --scanflags <value> <target>
```

Each probe in the next section sets one bit at a time, walking the flag field upward from 0x00.

Single-Flag Probes

NULL — 0x00 (no bits set)

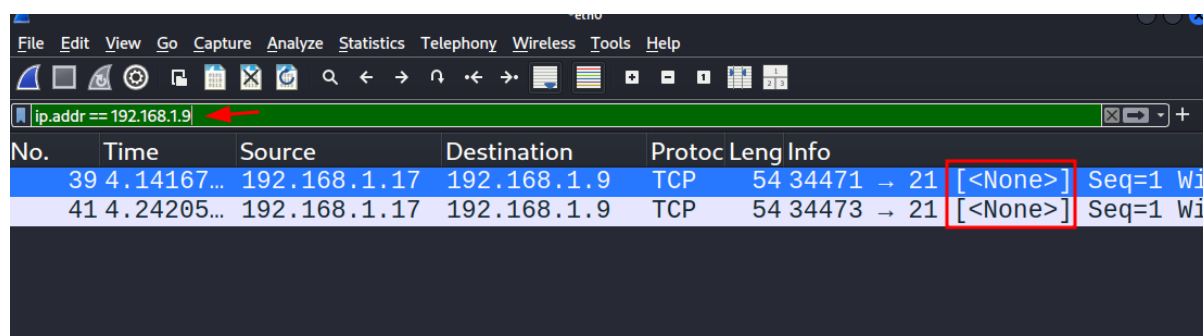
A NULL probe clears all control bits and sends a header with no flags set. Against port 21, the firewall returns nothing, so Nmap records the port as filtered.

```
nmap -p21 --scanflags 0x00 192.168.1.9
```

```
root@kali ~
└─> nmap -p21 --scanflags 0x00 192.168.1.9
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 06:50 -0400
Nmap scan report for 192.168.1.9
Host is up (0.00043s latency).

PORT      STATE      SERVICE
21/tcp    filtered  ftp
MAC Address: 00:0C:29:85:D1:03 (VMware)
```

Wireshark confirms the empty flag field: the Info column reports [**<None>**] for each outbound segment to the target.



FIN — 0x01 (bit 0)



Setting bit 0 raises FIN, the flag that normally signals the orderly end of a session. Sent cold against port 21, the lone FIN draws no reply through the firewall, and Nmap again reports filtered.

```
nmap -p21 --scanflags 0x01 192.168.1.9
```

```
root@kali ~  
└─> nmap -p21 --scanflags 0x01 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 06:51 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00046s latency).  
  
PORT      STATE      SERVICE  
21/tcp    filtered  ftp  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.85 seconds
```

The capture isolates a single **[FIN]** bit on each probe.

No.	Time	Source	Destination	Protoc	Leng	Info
30	3.84191...	192.168.1.17	192.168.1.9	TCP	54	38033 → 21 [FIN] Seq=1 W
31	3.94235...	192.168.1.17	192.168.1.9	TCP	54	38035 → 21 [FIN] Seq=1 W

SYN — 0x02 (bit 1)

Bit 1 carries SYN, the flag that opens a TCP handshake. Port 22 answers, so this probe behaves like the start of a genuine connection, and Nmap marks the service open.

```
nmap -p22 --scanflags 0x02 192.168.1.9
```



```
root@kali ~  
└─> nmap -p22 --scanflags 0x02 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 06:53 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00041s latency).  
  
PORT      STATE SERVICE  
22/tcp    open  ssh  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.68 seconds
```

The capture records the full exchange: the attacker's **[SYN]**, the target's **[SYN, ACK]**, and the attacker's **[RST]** that tears the half-open connection back down.

No.	Time	Source	Destination	Protoc	Leng	Info
23	2.88700...	192.168.1.17	192.168.1.9	TCP	58	33452 → 22 [SYN] Seq=0 Wi
24	2.88762...	192.168.1.9	192.168.1.17	TCP	60	22 → 33452 [SYN, ACK] Seq
25	2.88774...	192.168.1.17	192.168.1.9	TCP	54	33452 → 22 [RST] Seq=1 Wi

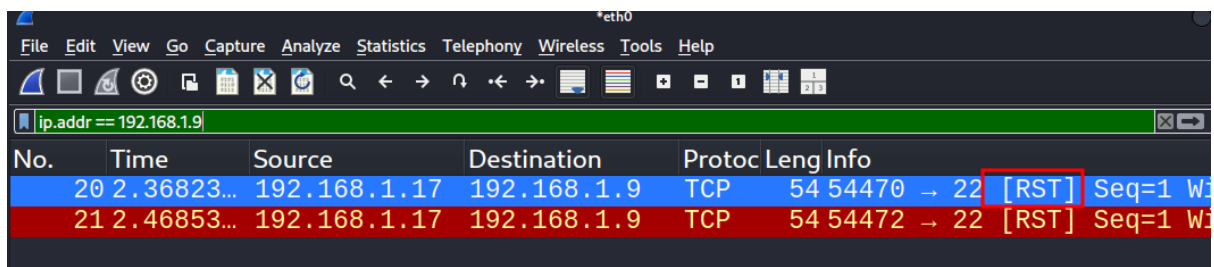
RST — 0x04 (bit 2)

Bit 2 sets RST, which normally aborts a connection. A bare reset to port 22 produces no useful response, and Nmap reports filtered.

```
nmap -p22 --scanflags 0x04 192.168.1.9
```

```
root@kali ~  
└─> nmap -p22 --scanflags 0x04 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 06:55 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00051s latency).  
  
PORT      STATE      SERVICE  
22/tcp    filtered  ssh  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.87 seconds  
root@kali ~
```

Wireshark shows the single **[RST]** bit on each probe.



No.	Time	Source	Destination	Protoc	Leng	Info
20	2.36823...	192.168.1.17	192.168.1.9	TCP	54	54470 → 22 [RST] Seq=1 W
21	2.46853...	192.168.1.17	192.168.1.9	TCP	54	54472 → 22 [RST] Seq=1 W

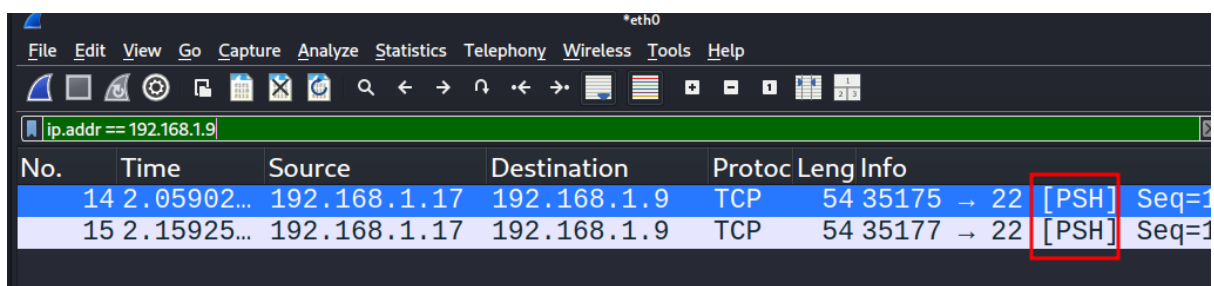
PSH — 0x08 (bit 3)

Bit 3 raises PSH, which tells the receiver to hand buffered data to the application at once. On its own it carries no connection semantics, so the target stays silent and Nmap records filtered.

```
nmap -p22 --scanflags 0x08 192.168.1.9
```

```
root@kali ~  
└─> nmap -p22 --scanflags 0x08 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 06:56 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00052s latency).  
  
PORT      STATE      SERVICE  
22/tcp    filtered  ssh  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.86 seconds  
root@kali ~
```

The capture isolates the single [PSH] bit.



No.	Time	Source	Destination	Protoc	Leng	Info
14	2.05902...	192.168.1.17	192.168.1.9	TCP	54	35175 → 22 [PSH] Seq=1
15	2.15925...	192.168.1.17	192.168.1.9	TCP	54	35177 → 22 [PSH] Seq=1

ACK — 0x10 (bit 4)



Bit 4 sets ACK, the acknowledgement flag present in every established segment. An unsolicited ACK violates TCP state, so the target answers with a reset; Nmap reads that reset as a closed port.

```
nmap -p22 --scanflags 0x10 192.168.1.9
```

```
root@kali ~  
└─> nmap -p22 --scanflags 0x10 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 06:57 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00045s latency).  
  
PORT      STATE SERVICE  
22/tcp    closed ssh  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.68 seconds  
root@kali ~
```

The capture pairs the attacker's [ACK] with the target's [RST] reply — the reset that drives the closed verdict.

No.	Time	Source	Destination	Protoc	Leng	Info
112	6.86162...	192.168.1.17	192.168.1.9	TCP	54	51652 → 22 [ACK] Seq=1 A
113	6.86224...	192.168.1.9	192.168.1.17	TCP	60	22 → 51652 [RST] Seq=1 W

URG — 0x20 (bit 5)

Bit 5 raises URG, which flags urgent data through the urgent pointer. Sent alone, it elicits no response, and Nmap reports filtered.

```
nmap -p22 --scanflags 0x20 192.168.1.9
```



```
root@kali ~  
└─> nmap -p22 --scanflags 0x20 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 06:58 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00043s latency).  
  
PORT      STATE      SERVICE  
22/tcp    filtered  ssh  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.87 seconds  
root@kali ~
```

Wireshark confirms a single **[URG]** bit on each probe.

The image shows a Wireshark packet capture on interface eth0. The filter is 'ip.addr == 192.168.1.9'. Two TCP packets are visible, both from 192.168.1.17 to 192.168.1.9 on port 22. The first packet (No. 118) has a sequence number of 7.19957... and the second (No. 132) has 7.29978... Both packets have the URG flag set, which is highlighted with a red box in the original image.

No.	Time	Source	Destination	Protoc	Leng	Info
118	7.19957...	192.168.1.17	192.168.1.9	TCP	54	53950 → 22 [URG] Seq
132	7.29978...	192.168.1.17	192.168.1.9	TCP	54	53952 → 22 [URG] Seq

Combining Flags with Bitwise OR

Because each flag owns a unique bit, combining flags is simply a bitwise OR — equivalently, adding their values into one byte. The four combinations below stack progressively more bits, and several of them mix flags that no legitimate session would set together, which is exactly what makes them useful for probing a firewall's handling of anomalous traffic.

FIN + PSH + URG — 0x29

The classic example raises FIN, PSH, and URG together: $0x01 \mid 0x08 \mid 0x20 = 0x29$ (41 decimal), the flag pattern of an Xmas scan. The breakdown below sums the three flags into one header byte.



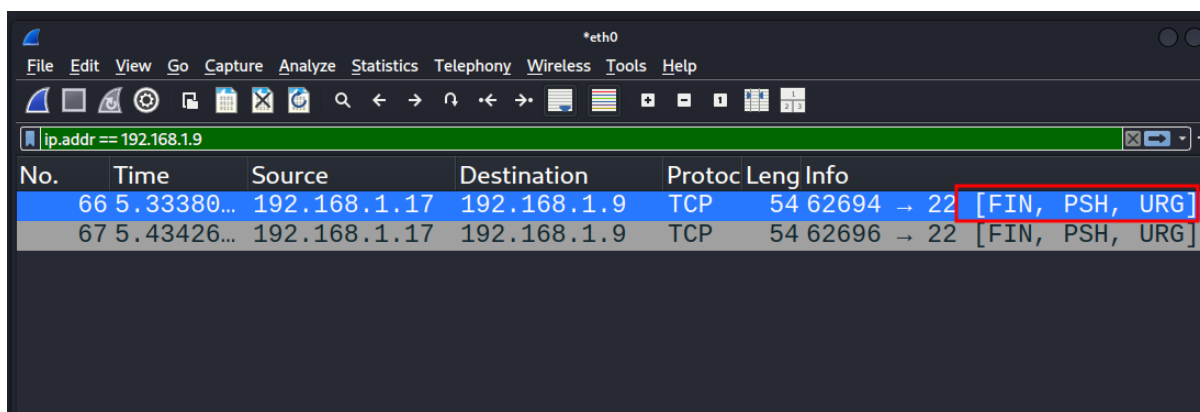
FLAGS	HEXADECIMAL	DECIMAL VALUE
FIN	0x01	1
PUSH	0x08	8
URG	0x20	32
Total	0x29	41

Feeding 0x29 to **--scanflags** sets all three bits in one probe. Against port 22 the target stays silent, and Nmap reports filtered.

```
nmap -p22 --scanflags 0x29 192.168.1.9
```

```
root@kali ~  
└─> nmap -p22 --scanflags 0x29 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 06:59 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00044s latency).  
  
PORT      STATE      SERVICE  
22/tcp    filtered  ssh  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.88 seconds
```

Wireshark decodes the combined byte back into its named flags, showing **[FIN, PSH, URG]** on each probe — proof that the hex value mapped to exactly the bits intended.

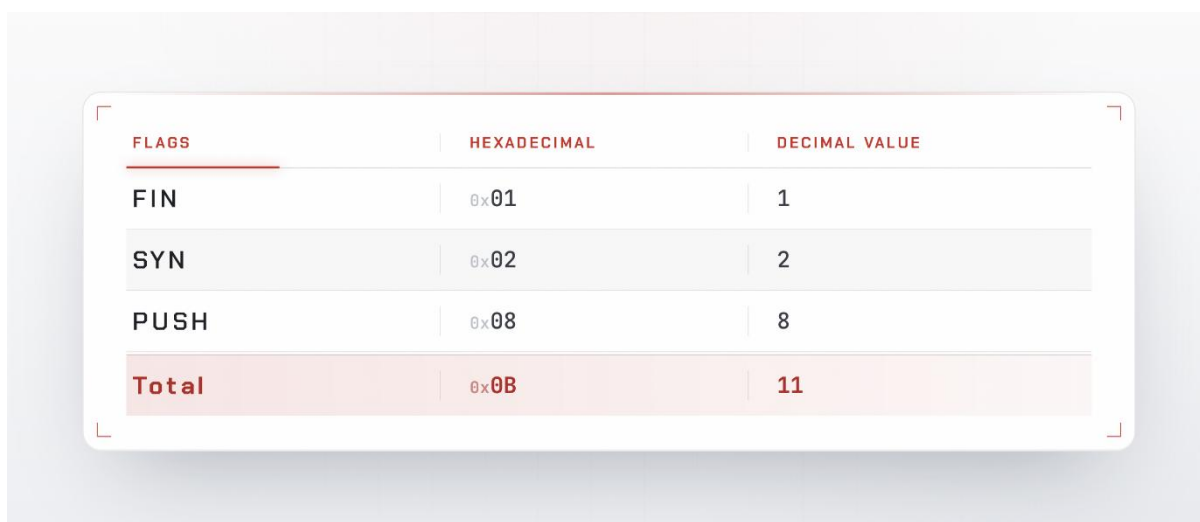


The image shows a Wireshark packet capture on the interface *eth0. The filter is set to ip.addr == 192.168.1.9. Two packets are visible:

No.	Time	Source	Destination	Protoc	Leng	Info
66	5.33380...	192.168.1.17	192.168.1.9	TCP	54	62694 → 22 [FIN, PSH, URG]
67	5.43426...	192.168.1.17	192.168.1.9	TCP	54	62696 → 22 [FIN, PSH, URG]

FIN + SYN + PSH — 0x0B

Moving into the low nibble, FIN, SYN, and PSH combine as 0x01 | 0x02 | 0x08 = 0x0B (11 decimal). The breakdown below shows the resulting 1011 pattern.



FLAGS	HEXADECIMAL	DECIMAL VALUE
FIN	0x01	1
SYN	0x02	2
PUSH	0x08	8
Total	0x0B	11

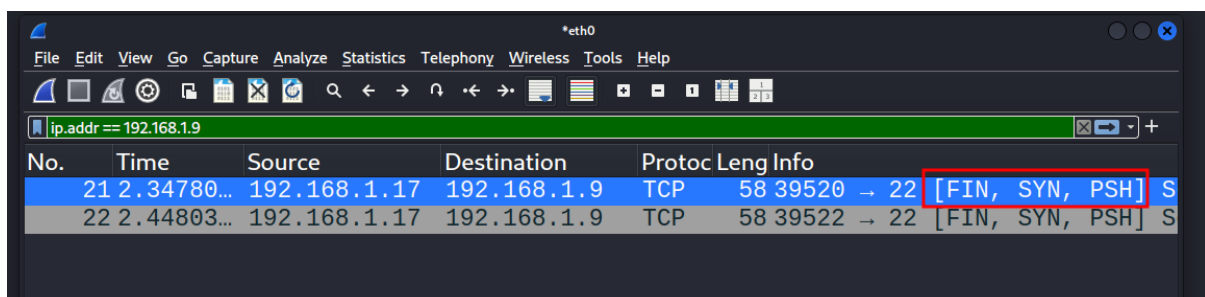
Running the probe sets all three bits at once, the target again stays silent, and Nmap reports filtered.

```
nmap -p22 --scanflags 0x0B 192.168.1.9
```



```
root@kali ~  
└─> nmap -p22 --scanflags 0x0B 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 07:00 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00036s latency).  
  
PORT      STATE      SERVICE  
22/tcp    filtered  ssh  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.90 seconds  
root@kali ~
```

Wireshark decodes the byte as **[FIN, SYN, PSH]** on each outbound segment.



FIN + RST + PSH — 0x0D

Swapping SYN for RST gives 0x01 | 0x04 | 0x08 = 0x0D (13 decimal), the 1101 pattern.
Pairing RST with FIN and PSH is meaningless to a healthy TCP stack, so the firewall drops it and Nmap reports filtered.

FLAGS	HEXADECIMAL	DECIMAL VALUE
FIN	0x01	1
RST	0x04	4
PUSH	0x08	8
Total	0x0D	13



```
nmap -p22 --scanflags 0x0D 192.168.1.9
```

```
root@kali ~  
└─> nmap -p22 --scanflags 0x0D 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 07:02 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00037s latency).  
  
PORT      STATE      SERVICE  
22/tcp    filtered  ssh  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.87 seconds  
root@kali ~
```

The capture resolves the byte to **[FIN, RST, PSH]**, matching the breakdown bit for bit.

No.	Time	Source	Destination	Protoc	Leng	Info
37	2.20736...	192.168.1.17	192.168.1.9	TCP	54	59690 → 22 [FIN, RST, PSH] S
38	2.30769...	192.168.1.17	192.168.1.9	TCP	54	59692 → 22 [FIN, RST, PSH] S

FIN + SYN + RST + PSH — 0x0F

Adding the fourth low-order flag fills the nibble: 0x01 | 0x02 | 0x04 | 0x08 = 0x0F (15 decimal). This is the most anomalous probe of the set, since SYN, RST, and FIN cannot coexist in any legitimate segment. The target offers no reply, and Nmap reports filtered.



FLAGS	HEXADECIMAL	DECIMAL VALUE
FIN	0x01	1
SYN	0x02	2
RST	0x04	4
PUSH	0x08	8
Total	0x0F	15

```
nmap -p22 --scanflags 0x0F 192.168.1.9
```

```
root@kali ~  
└─> nmap -p22 --scanflags 0x0F 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 07:03 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00054s latency).  
  
PORT      STATE      SERVICE  
22/tcp    filtered  ssh  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.87 seconds  
root@kali ~
```

Wireshark confirms the full low nibble, decoding the byte as **[FIN, SYN, RST, PSH]** on every probe.

No.	Time	Source	Destination	Protoc	Leng	Info
17	1.07582...	192.168.1.17	192.168.1.9	TCP	58	57150 → 22 [FIN, SYN, RST, PSH]
18	1.17632...	192.168.1.17	192.168.1.9	TCP	58	57152 → 22 [FIN, SYN, RST, PSH]



Consolidated Results

The image below consolidates every probe in the study, the byte it placed on the wire, the port targeted, and the state Nmap inferred from the target's response.

FLAG SET	HEX	DEC	PORT	OBSERVED STATE
NULL	0x00	0	21	filtered
FIN	0x01	1	21	filtered
SYN	0x02	2	22	open
RST	0x04	4	22	filtered
PSH	0x08	8	22	filtered
ACK	0x10	16	22	closed
URG	0x20	32	22	filtered
FIN, PSH, URG	0x29	41	22	filtered
FIN, SYN, PSH	0x0B	11	22	filtered
FIN, RST, PSH	0x0D	13	22	filtered
FIN, SYN, RST, PSH	0x0F	15	22	filtered

Three patterns stand out. The SYN probe opened a real handshake and revealed an open service; the ACK probe drew a reset that exposed a closed port; and every other single flag and combination landed on filtered, because the firewall silently dropped probes that no legitimate handshake would send. The lesson for an analyst is that **--scanflags** grants bit-level control of the TCP header, Wireshark verifies that control end to end, and the target's reaction — a reply, a reset, or silence — reveals both port state and firewall posture.

Mitigation Strategies

The same flag behaviour that an attacker probe is something a defender can lock down. The goal is to deny packets that no legitimate session produces, to drop traffic that falls outside an established connection, and to detect scanning activity before it yields useful information.



Drop illegal and anomalous flag combinations

A host firewall should discard packets whose flag combinations cannot occur in normal TCP, such as a header with no flags, every flag set, or mutually exclusive flags like SYN with FIN or SYN with RST. The iptables rules below drop the exact patterns exercised in this study before they ever reach a service.

- iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP
 - iptables -A INPUT -p tcp --tcp-flags ALL ALL -j DROP
 - iptables -A INPUT -p tcp --tcp-flags ALL FIN,PSH,URG -j DROP
 - iptables -A INPUT -p tcp --tcp-flags SYN,FIN SYN,FIN -j DROP
 - iptables -A INPUT -p tcp --tcp-flags SYN,RST SYN,RST -j DROP
- Enforce stateful filtering

A stateful firewall tracks connection state and rejects segments that arrive outside a known session, which neutralises bare FIN, RST, PSH, and ACK probes regardless of their exact flag pattern. Dropping packets in the INVALID connection state is a compact way to achieve this.

```
iptables -A INPUT -m conntrack --ctstate INVALID -j DROP
```

Detect, rate-limit, and reduce exposure

Layered controls catch what filtering alone cannot. An intrusion-detection system such as Snort or Suricata can alert on the anomalous flag patterns and port sweeps shown here, while connection rate-limiting and tools like fail2ban slow an attacker who probes repeatedly. Finally, reducing the attack surface remains the most durable defence: close unused ports, expose sensitive services such as SSH only to trusted networks or through a bastion host, keep the TCP stack and services patched, and log dropped packets so that scanning patterns surface in monitoring.

Conclusion

Crafting TCP probes by hand turns Nmap from a fixed-recipe scanner into a precise packet builder. By setting the control-bit field one bit at a time and combining bits into bytes up to 0x0F, this study showed how a target distinguishes an open port from a closed or filtered one, and how Wireshark confirms each hexadecimal value maps to the intended flags. The target's responses — a SYN that opened a service, an ACK that provoked a revealing reset, and a firewall that silently swallowed anomalous combinations — trace a direct line from packet contents to security posture. That same understanding cuts both ways: defenders who drop illegal flag combinations, enforce stateful filtering, and monitor for scan patterns close the very gaps that --scanflags exploits. With a single byte standing between an analyst and 256 possible flag combinations, fluency in the TCP control field remains essential for attackers and defenders alike.

FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

